

Web Scraping and Data Collection

Day 2: From Raw Output to Usable Data

Moses Boudourides

*Faculty, Graduate Program on Data Science
Northwestern University*

Moses.Boudourides@northwestern.edu

Moses.Boudourides@gmail.com

instats Seminar

Thursday, May 14, 2026
4:00 PM – 5:30 PM UTC

Day 2: From Raw Output to Usable Data

- 1 Why Raw Output Is Never Analysis-Ready
- 2 Cleaning as Measurement and Reproducibility
- 3 Five Common Problems in Raw Web-Derived Data
- 4 Applications: Cleaning Walkthroughs
- 5 The Streamlit Cleaning Module
- 6 Documentation: Logs and Metadata
- 7 Validation in Practice
- 8 Outcomes and Day 3 Preview

Interactive App

- **URL:** [Click here to open the Interactive Streamlit App](#)
- **No Installation:** Runs entirely in the browser via Streamlit — no Python or R required.
- **Navigation:** Use the sidebar to switch between the three seminar days.
- **Step-by-Step:** Each page walks through the core extraction, cleaning, and exploration computations for that day.
- **Applications:** Covers six real-world datasets from Health, Life, and Social Sciences — ClinicalTrials.gov, WHO Global Health Observatory, NIH RePORTER, U.S. Congress bills, World Bank environmental indicators, and GBIF biodiversity records.
- **Consistency:** Results match the companion Python and R scripts available on the seminar GitHub page.

Web-Based Data Are Rarely Analysis-Ready at the Moment of Extraction

Web pages were not designed with a research schema in mind. Relevant information may appear under inconsistent labels, in mixed text blocks, across multiple pages, or behind interactive elements. Common problems in raw outputs include:

- Inconsistent labels and category names
- Duplicated records from multiple access paths
- Irregular date formats across entries
- Missing values that may signal absent information or extraction failure
- Fields that need to be split, merged, or standardized

Every Cleaning Operation Is a Decision About What the Data Should Represent

Cleaning should not be understood as a purely technical step. Each operation encodes a **measurement decision**:

- Removing duplicates assumes that two records with the same identifier represent the same entity
- Standardizing date formats assumes a particular temporal resolution is appropriate
- Harmonizing labels assumes that variant spellings refer to the same category
- Dropping missing values assumes that records without a key field are not usable

These decisions should be **documented and justified**, not applied silently.

Reproducible Cleaning Requires Processing Logs and Preserved Raw Files

Dogucu and Çetinkaya-Rundel (2021) show that data acquisition from the web naturally introduces students to the realities of messy data and to the importance of reproducible preparation. In research settings, cleaning should be documented through:

- Processing logs (what operations were applied, in what order)
- Metadata sheets (field definitions, source URL, collection date, row count)
- Preserved versions of both raw and cleaned files

"That transformation should be documented. Processing logs, metadata sheets, preserved versions of raw and cleaned files, and explicit notes on parsing assumptions help maintain transparency and support later replication." — Boudourides (2025)

Validation Has at Least Five Distinct Dimensions in Web-Derived Data

Dimension	Description
Structural	Extraction captured intended units and fields consistently across pages
Semantic	Values in a field mean the same thing across observations
Coverage	Source population and extracted sample align for the research purpose
Cross-source	Records can be triangulated against external benchmarks
Analytic	Data behave plausibly in summaries, distributions, and cross-checks

These dimensions often require **iteration**: researchers may need to revise field definitions or cleaning rules once inconsistencies become visible.

Label Inconsistency Arises When the Same Category Appears Under Different Names

In the ClinicalTrials.gov data, the 'OverallStatus' field may contain values such as 'RECRUITING', 'Completed', 'not yet recruiting', and 'Active, not recruiting' — all referring to distinct but related concepts, expressed in different cases and formats.

Cleaning strategy:

- Uppercase all status values for consistency
- Map variant spellings to a controlled vocabulary
- Document the mapping decisions in a processing log

Directories Often Display the Same Entity Through Multiple Access Paths

Duplicate records arise when a web source lists the same entity under multiple categories, search results, or pagination paths. Deduplication requires:

- 1 Identifying a **unique identifier** (e.g., NCT ID for clinical trials, package ID for bills)
- 2 Deciding which duplicate to retain (first occurrence, most complete record, most recent)
- 3 Documenting the deduplication rule

In the absence of a stable identifier, deduplication must rely on combinations of fields — a less reliable approach that should be noted in the metadata.

Date Fields Frequently Vary in Format Across Records and Sources

Date inconsistency is one of the most common problems in web-derived data. A single source may contain dates in formats such as '2023-01-15', 'January 15, 2023', '15/01/2023', or '2023'. Standardization requires:

- Parsing all variants into a single format (ISO 8601: 'YYYY-MM-DD' is recommended)
- Deciding how to handle partial dates (year only, year-month only)
- Flagging records where date parsing failed

In the GovInfo congressional bills data, 'dateIssued' is already in ISO format — but this is the exception rather than the rule.

Missing Values May Signal Absent Information, Extraction Failure, or Source Norms

Missing values in web-derived data have at least three distinct causes:

- ➊ **Absent information:** the source genuinely does not contain the value (e.g., a trial without a registered sponsor)
- ➋ **Extraction failure:** the field was present but the parser did not capture it (e.g., a dynamically loaded element)
- ➌ **Source-specific reporting norms:** some registries do not require certain fields

These causes have different implications. Absent information may be substantively meaningful. Extraction failure should prompt re-inspection of the source. Source norms affect coverage comparability across registries or jurisdictions.

API Responses Embed Related Entities Within Records Rather Than Flat Tables

JSON responses from APIs such as NIH RePORTER and ClinicalTrials.gov embed related entities as nested objects:

```
{  
  "organization": {  
    "org_name": "Stanford University",  
    "org_city": "Stanford"  
  }  
}
```

Flattening these nested structures into tabular columns requires explicit decisions about which sub-fields to extract and how to name them. Fields that are themselves arrays (e.g., a list of conditions) must be joined into a string or handled as multi-valued attributes.

Flattening Nested JSON Reveals the Full Scope of Cleaning Required

The Day 2 script for ClinicalTrials.gov performs the following operations on the raw JSON cached on Day 1:

- 1 Navigate the nested 'protocolSection → identificationModule' path to extract 'NCTId' and 'BriefTitle'
- 2 Extract 'OverallStatus' from 'statusModule' and uppercase it for consistency
- 3 Extract 'LeadSponsor.name' from 'sponsorCollaboratorsModule', defaulting to "Unknown" when absent
- 4 Join the 'conditions' array into a comma-separated string
- 5 Drop duplicate rows by 'NCTId'

The result is a clean, flat CSV with one row per trial and five consistently populated columns.

Column Selection, Renaming, and Filtering Transform the WHO Response

The WHO GHO API returns records with technical column names ('SpatialDim', 'TimeDim', 'NumericValue') that must be renamed for readability. The Day 2 cleaning script:

- 1 Selects the relevant subset of columns from the raw response
- 2 Renames columns to 'CountryCode', 'Year', 'Sex', and 'LifeExpectancy'
- 3 Drops rows where 'LifeExpectancy' is missing
- 4 Rounds numeric values to one decimal place

The result reveals that each country-year combination appears three times — once for each sex category ('SEX_MLE', 'SEX_FMLE', 'SEX_BTSEX') — a structural feature that must be understood before any analysis.

Nested Organization and Agency Fields Require Explicit Extraction Logic

The NIH RePORTER API embeds organization and agency information as nested objects within each grant record. The Day 2 script:

- 1 Iterates over each record and safely extracts 'organization.org_name' and 'agency_ic_admin.name' with fallback defaults
- 2 Converts 'award_amount' to numeric, replacing null values with zero
- 3 Produces a clean CSV with five columns: 'ProjectNumber', 'Title', 'Organization', 'Agency', 'AwardAmount'

The resulting dataset enables straightforward analysis of funding distribution by organization and agency — a task that would be impossible from the raw nested JSON.

String Cleaning and Date Standardization Prepare Legislative Records for Analysis

The GovInfo API returns bill records with minimal fields but several cleaning needs. The Day 2 script:

- 1 Selects 'packageld', 'title', 'congress', 'datelssued', and 'docClass'
- 2 Strips extra whitespace from titles using 'str.strip()' (Python) / 'str_squish()' (R)
- 3 Parses 'datelssued' into a standard 'YYYY-MM-DD' date format
- 4 Renames columns to 'BillID', 'Title', 'Congress', 'Datelssued', 'DocumentClass'

This produces a clean, analysis-ready table of recent legislative bills with consistent field names and date formatting.

A New Source Introduced on Day 2 Demonstrates the Full Extract-Clean Pipeline

Application 5 introduces a new source — the World Bank API — to demonstrate the complete extract-then-clean pipeline in a single session.

The script:

- 1 Fetches population data for all countries in 2020 ('SP.POP.TOTL' indicator)
- 2 Parses the nested two-element list response (metadata + data records)
- 3 Extracts 'CountryCode', 'CountryName', 'Year', and 'Population'
- 4 Filters out aggregate regions (those without ISO country codes) and missing values

The result is a clean dataset of 95 country-level population records, ready for exploratory analysis on Day 3.

The App's Cleaning Module Makes Each Operation Configurable and Transparent

In the Streamlit app, the cleaning module exposes each operation as a configurable control:

Operation	Interface Control
Rename columns	Text input fields mapped to original column names
Standardize dates	Dropdown to select date format
Remove duplicates	Checkbox + column selector for unique key
Drop missing values	Column selector for required fields
Harmonize labels	Mapping table (original → standardized value)
Split columns	Separator input + new column name fields

Participants can preview the effect of each operation before applying it, and can compare raw and cleaned versions side by side.

Side-by-Side Comparison Makes the Value of Cleaning Concrete

A key pedagogical moment in Day 2 is the **side-by-side comparison** of raw and cleaned versions of the same dataset. This comparison reveals:

- How many records were dropped (and why)
- Which fields changed in format or content
- Whether the cleaning operations introduced any unintended changes
- What the final row count and column structure look like

This comparison is not merely a quality check — it is a **transparency mechanism** that documents the transformation from raw extraction to analysis-ready data.

Cleaning Addresses Formatting Problems but Cannot Resolve Structural Bias

Cleaning can standardize formats, remove duplicates, and fill missing values. It cannot fix:

- **Resource bias:** institutions with richer pages are over-represented
- **Incentive bias:** actors who selectively disclose information remain selectively represented
- **Coverage gaps:** entities not listed in the source are absent from the dataset
- **DOM-dependent extraction bias:** information that was never retrieved cannot be recovered

These limitations must be **acknowledged in the metadata** and discussed when interpreting results. A well-cleaned dataset is not necessarily a representative one.

A Processing Log Records Every Transformation Applied to the Raw Data

A processing log is a structured record of the cleaning operations applied to a dataset. It should include:

- Source URL and collection date
- Raw record count and cleaned record count
- List of operations applied (in order), with parameters
- Number of records dropped at each step and reason
- Field renaming map
- Notes on parsing assumptions or ambiguous decisions

The Streamlit app generates a processing log automatically and makes it available for download alongside the cleaned dataset.

A Metadata Sheet Describes the Dataset for Future Users and Replication

A metadata sheet complements the processing log by describing the **final state** of the dataset:

Field	Content
Dataset name	Descriptive title
Source	URL(s) and API endpoint(s)
Collection date	Date of original extraction
Unit of analysis	What one row represents
Row count	Number of records in cleaned file
Column definitions	Name, type, and description of each field
Known limitations	Coverage gaps, bias, missing fields

Preserving Raw Files Enables Replication and Error Detection

A reproducible workflow preserves **both** the raw extracted file and the cleaned output. This is important because:

- The raw file allows independent replication of the cleaning steps
- Errors in cleaning can be detected and corrected without re-fetching the source
- The raw file documents what the source looked like at the time of collection (sources change)
- Reviewers and collaborators can verify that cleaning decisions were appropriate

In the seminar repository, raw files are stored in 'data/cache/' and excluded from the GitHub repository via '.gitignore'. Participants are encouraged to archive raw files locally.

Structural Validation Confirms That Extraction Was Consistent Across All Pages

Structural validation asks: did the extraction process capture the intended units and fields consistently? Practical checks include:

- Comparing the number of extracted records against the expected total from the source
- Sampling random records and verifying them against the source page
- Checking that all required fields are populated across all records
- Logging extraction coverage (pages processed, records per page)

In the seminar scripts, a simple record count is printed after each extraction. More rigorous validation would compare against known totals from the source's own metadata.

Semantic Validation Confirms That Field Values Mean the Same Thing Across Records

Semantic validation asks: do the values in a field mean the same thing across all observations? Practical checks include:

- Reviewing the distribution of categorical fields for unexpected values
- Checking numeric fields for implausible values (negative ages, future dates, zero award amounts)
- Verifying that date fields fall within the expected range
- Comparing a sample of records against their source pages to confirm field interpretation

The Streamlit app's Explore module supports these checks through summary tables, value counts, and missingness reports.

Cross-Source Validation Compares Extracted Records Against External Benchmarks

Cross-source validation compares extracted records against an independent source to assess coverage and accuracy. Examples:

- Comparing the number of trials extracted from ClinicalTrials.gov against the registry's own reported totals
- Comparing country-level population figures from the World Bank against UN Population Division estimates
- Comparing legislative bill counts from GovInfo against Congress.gov's own statistics

Cross-source validation is not always feasible, but it is the strongest form of validation available for web-derived data.

Analytic Validation Checks Whether Cleaned Data Produce Plausible Results

Analytic validation asks: do the data behave plausibly in summaries, distributions, and cross-checks with outside information? For the seminar datasets:

- Life expectancy values should fall within the range of approximately 50–90 years
- Award amounts for NIH grants should be in the range of tens of thousands to millions of dollars
- Population figures should be consistent with known country sizes
- Bill counts should reflect the pace of legislative activity in the relevant session

Implausible values may indicate extraction errors, cleaning mistakes, or genuine source anomalies — all of which deserve investigation.

R Scripts Use dplyr and purrr to Express the Same Cleaning Logic Declaratively

The R cleaning scripts use the tidyverse ecosystem to express the same operations as the Python scripts:

- `'dplyr::select()'`, `'rename()'`, `'filter()'`, `'mutate()'` for column operations
- `'purrr::map_dfr()'` for iterating over nested JSON lists and binding rows
- `'stringr::str_squish()'` for whitespace normalization
- `'tidyr::replace_na()'` for missing value handling

The R approach is more declarative and pipeline-oriented than the Python approach, making the sequence of operations easier to read — an advantage for teaching purposes.

By End of Day 2, Participants Understand How Raw Data Becomes Usable Data

By the end of Day 2, participants should be able to:

- Identify the most common problems in raw web-derived data
- Apply no-code cleaning operations through the Streamlit app
- Understand why each cleaning operation is a measurement decision
- Compare raw and cleaned versions of a dataset and explain the differences
- Generate a processing log and metadata sheet for a cleaned output
- Recognize what cleaning can and cannot fix in terms of structural bias

Day 3 Enables Participants to Adapt the Workflow to Their Own Research Materials

Day 3 shifts the focus from shared examples to **participant-specific adaptation**. Key activities include:

- Generating summary tables, missingness checks, and simple visualizations from cleaned data
- Uploading participants' own URLs, URL lists, CSV files, or existing datasets
- Deciding which module of the pipeline to enter (collection, cleaning, or exploration)
- Interpreting preliminary outputs and assessing feasibility for their own research question
- A short but substantive discussion of the **ethics of web scraping and digital data collection**

References

- Boudourides, M. (2025). Web Scraping and Data Collection for Life and Social Sciences.
- Brown, M. A. et al. (2025). Web scraping for research: Legal, ethical, institutional, and scientific considerations. *Big Data & Society*, 12(1), 1–17.
- Dogucu, M. & Çetinkaya-Rundel, M. (2021). Web scraping in the statistics and data science curriculum. *Journal of Statistics and Data Science Education*, 29(2), 112–122.
- Grimmer, J., Roberts, M. E., & Stewart, B. M. (2022). *Text as Data*. Princeton University Press.
- Li, F., Zhou, Y., & Cai, T. (2021). Trails of data: Three cases for collecting web information for social science research. *Social Science Computer Review*, 39(5), 839–855.
- Ohme, J. et al. (2024). Digital trace data collection for social media effects research. *Communication Methods and Measures*, 18(2), 124–141.

Questions and Discussion

Thank you!

Questions?

`Moses.Boudourides@northwestern.edu`

`Moses.Boudourides@gmail.com`